Parte 1 de 1 Duração: 120 minutos Com consulta (folha A4 manuscrita)

Leia toda a prova antes de iniciar a resolução. Se sentir que algum detalhe está em falta volte a ler. Se ainda assim sentir que existe uma omissão assuma pressupostos razoáveis e esclareça-os de forma justificada na sua resposta. Evite solicitar a presença do vigilante, facto que incomodará os restantes colegas. Pretende-se que desenvolva um serviço (cliente e servidor) de *chat* (conversação) muito simplificado sobre UDP. É esperado que os clientes possam enviar frases (obtidas da *standard input* através do `fgets()`) com, no máximo, 49 carateres. Uma frase é um conjunto de caracteres terminado por '`\n`'. A frase "`quit\n`" deve ser interpretada pela aplicação cliente como um pedido do utilizador para terminar imediatamente a sua execução. Como resposta a uma frase recebida, o servidor deve enviar sempre, e apenas ao seu remetente, uma única mensagem com as últimas frases recebidas (10 no máximo), incluindo a frase que esse cliente lhe acabou de enviar. No entanto cada uma dessas frases deve ser enriquecida, no seu início, com o endereço IP e porto (i.e. "coordenadas") do cliente que a enviou. Ou seja, o servidor deve de ir guardando localmente as frases recebidas num formato aumentado, para que comecem sempre pelas coordenadas do seu remetente e só depois tenham o conteúdo efetivamente recebido (caractere de mudança de linha '`\n`' incluído). O servidor não precisa que as mensagens que envia lhe sejam confirmadas. Os utilizadores colocam habitualmente o seu nome em todas as frases que enviam para simplificar a sua identificação. Tal não é relevante para a sua resolução, mas ajuda a compreender a seguinte ilustração de fluxo (repare que apenas depois de Ana enviar uma segunda frase recebe a primeira frase do João. Ou seja, quando alguém faz uma pergunta convém enviar, passado um tempo razoável, uma nova frase para receber do servidor a cópia atualizada do diálogo em curso.):

Emissor	Frase	Recetor
Cliente 1	"Hello, I'm Ana.\n"	Servidor
Servidor	"127.0.0.1:62199# Hello, I'm Ana.\n"	Cliente 1
Cliente 2	"Hello, I'm João.\n"	Servidor
Servidor	"127.0.0.1:62199# Hello, I'm Ana.\n 127.0.0.1:62200# Hello, I'm João.\n"	Cliente 2
Cliente 1	"Ana: is someone online?\n"	Servidor
Servidor	"127.0.0.1:62199# Hello, I'm Ana.\n 127.0.0.1:62200# Hello, I'm João.\n 127.0.0.1:62199# Ana: is someone online?\n"	Cliente 1
Cliente 1	"Ana: hello João. How are you?\n"	Servidor
Servidor	"127.0.0.1:62199# Hello, I'm Ana.\n 127.0.0.1:62200# Hello, I'm João.\n 127.0.0.1:62199# Ana: is someone online?\n 127.0.0.1:62199# Ana: hello João. How are you?\n"	Cliente 1
Cliente 2	"João: someone to talk to?\n"	Servidor
Servidor	"127.0.0.1:62199# Hello, I'm Ana.\n 127.0.0.1:62200# Hello, I'm João.\n 127.0.0.1:62199# Ana: is someone online?\n 127.0.0.1:62199# Ana: hello João. How are you?\n 127.0.0.1:62200# João: someone to talk to?\n"	Cliente 2

Nota: `#include <stdio.h>` // `char *fgets(char *str,int numChars,FILE *stream)` // To read from the standard input pass `stdin` as the stream. The `fgets` function reads a string from the input `stream` argument and stores it in `str`. `fgets` reads characters from the current stream position to and including the first newline character, to the end of the stream, or until the number of characters read is equal to `numChars-1`, whichever comes first. The result stored in is appended with a `NULL` character. The newline character, if read, is included in the string.

1) ***[35%]*** Desenvolva o cliente (veja a estrutura sugerida abaixo e as respetivas alíneas desta pergunta).
2) ***[50%]*** Desenvolva o servidor (veja a estrutura sugerida abaixo e as respetivas alíneas desta pergunta).
3) ***[10%]*** Reprograme o servidor (apresente apenas os trechos de código diferentes) para que este passe a "esquecer" todas as frases em memória sempre que passam mais de 5 minutos de inatividade dos clientes.
4) **[5%]** Explique (sem recorrer a código) como poderia fazer para que o servidor passasse a esquecer, não todas as frases, mas apenas as que possuem mais de cinco minutos de existência desde a sua receção.

```
/*========================== UDP Chat Client ==============================
Esta é a estrutura do código sugerida para a aplicação cliente. Exemplo de linha de comando usada para
contatar um servidor local UDP à escuta no porto 6000:
EN: This is the structure recommended to the client code. The command line used to contact a local server
listening at the 6000 UDP port is as follows:
   chat_client 127.0.0.1 6000
Todas as alíneas possuem a mesma cotação. EN: all items have the same weight.
================================================================================*/

#include <stdio.h>
#include <winsock.h>

void Abort(const char* msg);

/*_____________________________________ main ____________________________________
*/

int main(int argc, char* argv[])
{
        SOCKET sockfd;
        int iResult, phrase_len, nbytes;
        struct sockaddr_in serv_addr, cli_addr;
        WSADATA wsaData;

        // a) Declare a variável phrase destinada a guardar a frase introduzida pelo utilizador. Não esqueça
        //    que a mesma deve prever espaço para guardar o zero terminador de qualquer string. Sugere-se que
        //    use a mesma variável para guardar a resposta do servidor. Preveja uma dimensão adequada. Pode
        //    declarar e inicializar aqui outras variáveis que entender convenientes para a sua resolução.
        //    (EN: declare here the variable phrase that will store the phrase introduced by the user. Don't
        //    forget to anticipate room for the null terminator character. We recommend to also use this
        //    variable to receive the server's response. Define here and initialize other variables you need.

        /*..A..*/

        /*=============== WINSOCKS LIBRARY INITIALIZATION ==============*/
        iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
        if (iResult != 0) {
                printf("WSAStartup failed: %d\n", iResult);
                getchar();
                exit(1);
        }

        /*=============== CREATE A SOCKET TO SEND/RECEIVE DATAGRAMS ==============*/
        // b) Crie um socket UDP (EN: create an UDP socket).

        sockfd = /*..B..*/

        if (sockfd == INVALID_SOCKET)
                Abort("Unable to obtain a socket from operating system!");

        /*================= SETUP THE SERVER ADDRESS ====================*/
        memset((char*)&serv_addr, 0, sizeof(serv_addr)); /*Coloca a zero todos os bytes*/
        serv_addr.sin_family = AF_INET; /*Address Family: Internet*/

        // c) Preencha o campo relativo ao porto de destino pretendido, passado através da linha de comando
        //    (EN: set the server port number to the value provided as a command line argument).
        serv_addr.sin_addr.s_addr = inet_addr(argv[1]);

        serv_addr.sin_port = /*..C..*/;

        /*====================== SEND THE MESSAGE TO THE SERVER =================*/
        do {
                printf("Your phrase, please (press the ENTER key to terminate): ");
                fflush(stdout);
                // d) Utilize a função fgets() para ler a frase introduzida na stdin e guardar na variável
                //    definida phrase. Lembre-se que a frase a guardar deve estar limitada a um máximo de 49
                //    carateres, '\n' incluído. Se a frase colhida pelo fgets() não contemplar o '\n' (porque
```

```
		//    o utilizador introduziu mais que 48 caracteres) deve ter o cuidado de incluí-lo na
		//    última posição de phrase.
		//    (EN: use the function fgets() to read the phrase provided by the user at the stdin.
		//    Take care that the phrase must be limited to a maximum of 49 characters ('\n'
		//    included. Store it in the phrase variable. If the returned string does not include the
		//    '/n' (because the user provided more than 48 characters) remember to introduce it,
		//    yourself, as the last phrase character.

		/*..D..*/

		// e) O cliente deve terminar quando o utilizador introduzir a frase "quit\n"
		//    (EN: The client must exit when the user enters the message: "quit\n").

		/*..E..*/

		// f) Obtenha o número de bytes (comprimento) da frase
		// (EN: get the phrase length in bytes)

		phrase_len = /*..F..*/

		// g) Envie a frase introduzida para o servidor (EN: send the received phrase to the server).

		if(/*..G..*/ ) == SOCKET_ERROR)
			Abort("The operating system is not able to send the datagram.");

		/*====================== WAIT FOR THE ANSWER FROM THE SERVER ========================*/
		// h) Aguarde por resposta do servidor e coloque-a também na variável phrase
		// (EN: wait for the server answer and store it also into the same variable phrase).

		nbytes = /*..H..*/

		if (nbytes == SOCKET_ERROR) Abort("Error while receiving the server answer.");
		phrase[nbytes] = '\0';
		printf("#### On-going dialog ###\n%s\n", phrase);
	} while (1);

	/*======================== CLOSE THE SOCKET ============================*/
	closesocket(sockfd);
	exit(EXIT_SUCCESS);
}

/*_____________________________ Abort_____________________________________
  Mostra uma mensagem de erro e o código associado ao último erro com Winsocks.
  Termina a aplicação com "exit status" a 1 (constante EXIT_FAILURE).
  EN: Presents the last Winsocks message and code number of the last error reported.
  Exit the client with the EXIT_FAILURE error status.
_____________________________________________________________________________*/
void Abort(const char* msg){
	fprintf(stderr, "<CLI> Fatal error: <%s> (%d)\n", msg, WSAGetLastError());
	exit(EXIT_FAILURE);
}
```

```
/*============================== UDP Chat Server ==============================
Todas as alíneas possuem a mesma cotação. EN: all items have the same weight.
================================================================================*/

#include <stdio.h>
#include <winsock.h>

#define SERV_UDP_PORT 6000
#define BUFFERSIZE 100

void Abort(const char* msg);

/*__________________________________ main ____________________________________
*/

int main(int argc, char* argv[])
{
	SOCKET sockfd;
	int iResult, nbytes;
	int length_addr;
	struct sockaddr_in serv_addr, cli_addr;
	int i=0;
	WSADATA wsaData;
	// a) Declare um array que permita guardar as últimas 10 frases recebidas. Lembre-se que cada frase
	//    tem, no máximo, 49 caracteres. No entanto o servidor não guarda a frase tal qual é recebida
	//    (ex. "Hello, I'm Ana.\n") mas sim uma versão aumentada da mesma, que revela a localização do
	//    seu autor (ex. "127.0.0.1:62199# Hello, I'm Ana."). Este array deve funcionar como uma
	//    estrutura FIFO (First In First Out). Ou seja, quando for recebida a 11ª frase do diálogo, a 1ª
	//    recebida deve ser descartada por não caber; quando for recebida a 12ª deve descartar-se a 2ª; e
	//    assim sucessivamente. A mensagem que o servidor remete ao (último) cliente que o contata é
	//    composta pelas frases aumentadas alojadas nesse array, concatenadas na sequência em que foram
	//    recebidas. Inicialize o array de forma adequada. Defina/inicialize outras variáveis que entenda
	//    necessárias.
	//    (EN: declare an array to store the last 10 received phrases. Remember from the initial text is
	//    (e.g., "Hello, I'm Ana.") is actually stored in its augmented form (e.g., "127.0.0.1:62199# Olá
	//    sou a Ana.". This array must work as a FIFO (First In First Out) structure. For instance, when
	//    the eleventh (11th) phrase was received, the first one (1st) must be discard since the array has
	//    only ten positions. When the twelfth (12th) message was received the second one (2nd) must be
	//    discarded. And so on. Pay attention to the way the answer provided by the server must be built.
	//    The overall message is the set of the augmented phrases stored, concatenated in the sequence
	//    they were received. Initialize the array properly. Define / initialize other variables as
	//    needed.

	/*..A..*/

	// b) Declare uma string para guardar a concatenação das frases do array anterior. Esta string,
	//    devidamente atualizada, é a mensagem que o servidor envia a cada cliente que o contacte.
	//    EN: Declare a string to store the concatenation of the phrases stored in the previously defined
	//    array. This string, continuously updated, is be the message that the server use to answer each
	//    client contact.

	/*..B..*/

	/*=============== WINSOCKS LIBRARY INITIALIZATION ==============*/
	iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
	if (iResult != 0) {
		printf("WSAStartup failed: %d\n", iResult);
		getchar();
		exit(1);
	}

	/*============ CREATE A SOCKET TO SEND/RECEIVE DATAGRAMS ============*/
	// c) Crie um socket UDP (EN: create an UDP socket).

	if((sockfd = /*..C..*/) Abort("Unable to obtain a socket from operating system!");

	/*=============== BINDS THE SOCKET TO THE SERVER ADDRESS ===============*/
```

```
/*Anuncie ao sistema que pretende receber datagramas vindos de qualquer interface de
rede, no porto pretendido.
EN: Ask the operating system to receive datagrams from any network interface on a given port.
*/
memset((char*)&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET; /*Address Family: Internet*/
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);  /*Host TO Network Long*/

// d) Preencha o campo relativo ao porto local pretendido
//    (EN: setup the value of the field sin_port).

serv_addr.sin_port = /*..D..*/; //(SERV_UDP_PORT)

// e) Associe o socket ao porto pretendido
//  (EN: bind socket sockfd to the specified port).

if(/*..E..*/) == SOCKET_ERROR){

        // f) Caso tenha ocorrido um erro, associe o socket a um porto escolhido pelo sistema
        //    (EN: In case of error, attempt to use a system provided port).

        serv_addr.sin_port = /*..F..*/;
        if(/*..F..*/) == SOCKET_ERROR)
                Abort("Operating System is currently unable to bind the socket.");
}

/*================ ANSWER CLIENTS ITERACTIVELY =============*/
while (1) {
        fprintf(stderr, "<CHAT SERVER>Waiting new phrases from clients.\n");
        // g) Aguarde pela receção de um datagrama UDP no socket sockfd
        //    (EN: wait for a new datagram on socket sockfd).

        length_addr = sizeof(cli_addr);
        nbytes= /*..G..*/
        if (nbytes == SOCKET_ERROR) Abort("Error while waiting to receive a new datagram.");
        buffer[nbytes]='\0';

        // h) Utilize a função sprintf () para criar uma frase aumentada constituída na sua primeira
        //    parte pelo endereço do cliente que a enviou seguida da frase recebida, tal como definido
        //    no texto inicial. Pode utilizar a variável buffer para armazenar temporariamente esta
        //    frase aumentada.
        //    (EN: Use the function sprintf() or sprintf_s() to create an augmented phrase with the
        //    client's address concatenated to the received phrase it sent, as specified in the
        //    initial text. The variable buffer can be used to store it temporarily.

        sprintf(buffer, /*..H..*/,/*..H..*/);

        // i) Adicione a frase aumentada ao array definido em a). Este array deve funcionar como
        //    uma FIFO. Se guardar sempre a frase mais recente na primeira posição do array ([0]),
        //    deve, antes de a adicionar, deslocar as frases anteriores uma posição acima. Ou seja,
        //    a frase que ocupa a posição [8] transita para a posição [9], a que está em [7] passa
        //    para [8], etc.). Repare que, caso o array esteja efetivamente cheio, este processo
        //    conduzirá ao descarte (desejável) efetivo da frase mais antiga de entre as guardadas
        //    e que ocupará sempre a posição [9]. O código seguinte pressupõe que inicializou com uma
        //    string vazia ("") todas as posições do array.
        //    (EN: Add the message built in the previous question to the array defined in a). The
        //    array works as a FIFO. We suggest to store the most recent phrase in the first position
        //    of the array ([0]). Before adding a new arrived phrase all the current stored phrases
        //    should be shifted one position above in the array. The phrase stored at position [8]
        //    should be copied to the position [9], the one at [7] copied to [8], and so on.
        //    If the array is full, the older message (stored at [9]) is discarded as desired.
        //    The following code assumes the array was initialized with empty strings ("").

        for (i=9; i>-1; i--) strcpy(/*..I..*/);

        // j) Guarde na string declarada em b) a concatenação de todas as frases do array a).
        //    EN: Store in the string declared in b) the concatenation of all phrases stored in
```

```
                //    the array a).

                /*..J..*/

                // k) Envie a mensagem (a concatenação mencionada atrás) ao cliente que o contatou.
                //    (EN: send the message (the concatenation mencionada before) to the client that
                //    is contacting the server)

                /*..K..*/
}

/*_____________________________ Abort_____________________________________
Código igual ao da função com o mesmo nome que o cliente usa.
EN: The same code the Abort function used by the client uses.
_____________________________________________________________________________*/
```

Nota: Não esqueça de responder às perguntas 3 e 4 (releia a parte final da primeira página, por favor).
EN: Don't Forget to answer the questions 3 and 4 (check the bottom of the first page, please).